

PROGRAMAÇÃO MODULAR

POLIMORFISMO *AD HOC*

PROF. JOÃO CARAM

POLIMORFISMO

2

- Polimorfismo → muitas formas.
- Em POO: princípio que permite que classes filhas tenham métodos iguais, mas comportamentos diferentes.
 - Métodos iguais → mesma assinatura.
 - Comportamentos diferentes → ações diferentes.

POLIMORFISMOS

3

- Polimorfismo universal:

- de Inclusão.

- Paramétrico.

- Polimorfismo *ad hoc*¹:

- Coerção.

- Sobrecarga.

¹ - *ad hoc* làdóquel (locução latina que significa "para isso"): Que se destina a um fim específico

"*ad hoc*", in Dicionário Priberam da Língua Portuguesa [online], 2008-2021, <http://priberam.pt/dlpo/ad-hoc> [acesso em 21/04/2021].

4

SOBRECARGA (*OVERLOAD*)



SOBRECARGA (*OVERLOAD*)

5

- Criação de diversos métodos com o mesmo nome mas parâmetros diferentes.
 - Ações diferentes.
- Não é universal: somente para os tipos/classes para os quais foi implementada a sobrecarga.

A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

SOBRECARGA

Operadores

```
double x, y, z;
```

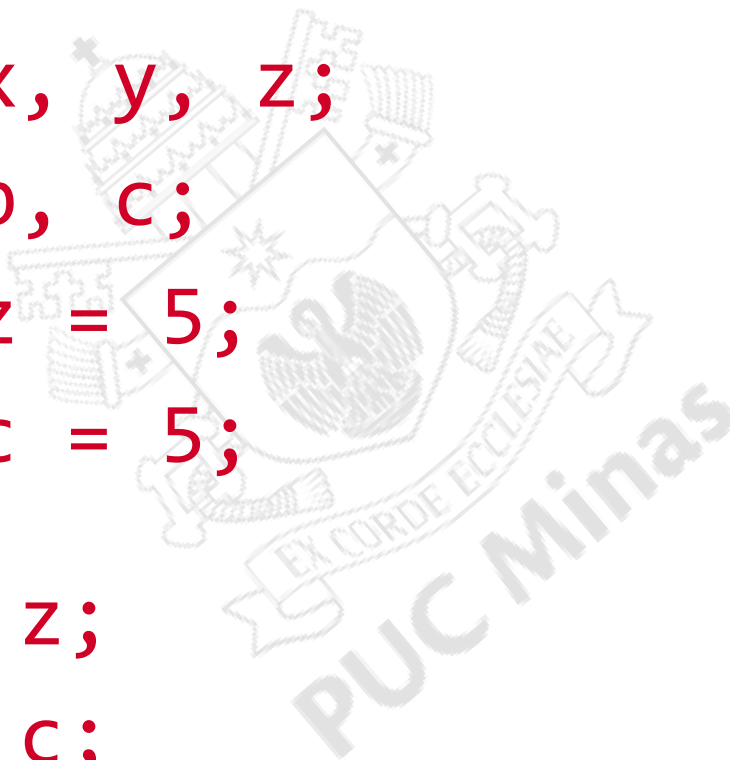
```
int a, b, c;
```

```
y = 5; z = 5;
```

```
b = 5; c = 5;
```

```
x = y + z;
```

```
a = b + c;
```



SOBRECARGA

Operadores

```
double x, y, z;
```

```
int a, b, c;
```

```
y = 5; z = 5;
```

```
b = 5; c = 5;
```

```
x = y + z; 10.0
```

```
a = b + c; 10
```


SOBRECARGA

Operadores

```
double x, y, z;
```

```
String s, u, v;
```

```
y = 5; z = 5;
```

```
u = "Objetos"; v = " e Classes";
```

```
x = y + z;
```

```
s = u + v;
```

SOBRECARGA

Operadores

```
double x, y, z;
```

```
String s, u, v;
```

```
y = 5; z = 5;
```

```
u = "Objetos"; v = " e Classes";
```

```
x = y + z;
```

```
s = u + v;
```

SOBRECARGA

Operadores

```
double x, y, z;
```

```
String s, u, v;
```

```
y = 5; z = 5;
```

```
u = "Objetos"; v = " e Classes";
```

```
x = y + z;
```

10.0

```
s = u + v;
```

"Objetos e Classes"

SOBRECARGA

Métodos

```
String nome = "João";  
int resposta = 42;  
Data hoje = new Data();  
System.out.println(nome);  
System.out.println(resposta);  
System.out.println(hoje);
```

SOBRECARGA

Métodos

```
String nome = "João";  
int resposta = 42;  
Data hoje = new Data();  
System.out.println(String);  
System.out.println(Integer);  
System.out.println(Object);
```

SOBRECARGA

Construtores no Java

```
public Pizza();  
public Pizza(int adicionais);  
public Pizza(int adicionais,  
             EBorda borda);
```

SOBRECARGA

Métodos

```
double maiorPreco(Comida a, comida b);  
double maiorPreco(Comida[] vet);  
double maiorPreco(List<Comida> lista);
```

16

COERÇÃO (*CAST*)



COERÇÃO (CAST)

17

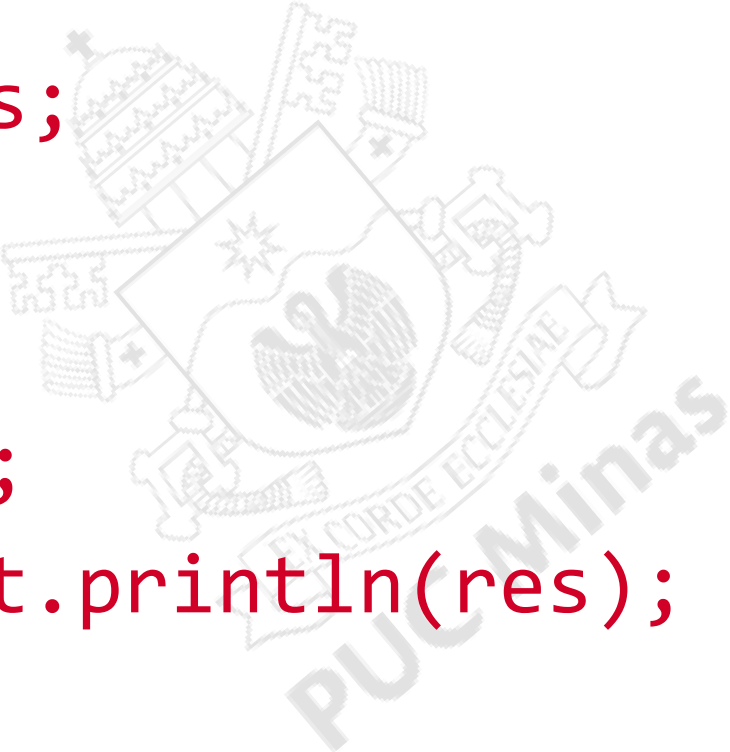
- Conversão de tipo explícita definida pelo programador.
- Só será aceita se houver alguma relação de parentesco entre as classes:
 - Tipos básicos.
 - Conversão classe mãe → classe filha.

A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

COERÇÃO (CAST)

```
int x, y;  
double res;  
x = 5;  
y = 2;  
res = x/y;  
System.out.println(res);
```



COERÇÃO (CAST)

```
int x, y;  
double res;  
x = 5;  
y = 2;  
res = x/y;
```

```
System.out.println(res);
```

2.0

COERÇÃO (CAST)

```
int x, y;  
double res;  
x = 5;  
y = 2;  
res = (double)x/y;  
System.out.println(res);
```

COERÇÃO (CAST)

```
int x, y;  
double res;  
x = 5;  
y = 2;  
res = (double)x/y;  
System.out.println(res);
```

2.5

COERÇÃO (CAST)

Comida comida;

Pizza pizza;

comida = new Pizza(4);

System.out.println(comida.precoFinal());

COERÇÃO (CAST)

Comida comida;

Pizza pizza;

comida = new Pizza(4);

System.out.println(**comida.precoFinal()**);

49

COERÇÃO (CAST)

Comida comida;

Pizza pizza;

comida = new Pizza(6);

System.out.println(**comida.desconto()**);



COERÇÃO (CAST)

```
Comida comida;
```

```
Pizza pizza;
```

```
comida = new Pizza(6);
```

```
pizza = (Pizza)comida;
```

```
System.out.println(pizza.desconto());
```

2.50

COERÇÃO (CAST)

Comida comida;

Pizza pizza;

Sanduiche sanduiche;

comida = new Pizza(6);

comida = new Sanduiche(0,true);

pizza = (Pizza)comida;

COERÇÃO (CAST)

Comida comida;

Pizza pizza;

Sanduiche sanduiche;

comida = new Pizza(6);

comida = new Sanduiche(0,true);

pizza = (Pizza)comida;



COERÇÃO (CAST)

Comida comida;

Pizza pizza;

Sanduiche sanduiche;

sanduiche = new Sanduiche(0,true);

pizza = (Pizza)sanduiche;



COERÇÃO (CAST)

@Override

```
public boolean equals(Object obj){  
    Pizza outra = (Pizza)obj;  
    return quantidadeIngredientes == outra.quantidadeIngredientes  
        && borda.equals(outra.borda);  
}
```

COERÇÃO (CAST)

@Override

```
public boolean equals(Object obj){  
    Pizza outra = (Pizza)obj;  
    return quantidadeIngredientes == outra.quantidadeIngredientes  
        && borda.equals(outra.borda);  
}
```

COERÇÃO (CAST)

@Override

```
public boolean equals(Object obj){  
    Pizza outra = (Pizza)obj;  
    return quantidadeIngredientes == outra.quantidadeIngredientes  
        && borda.equals(outra.borda);  
}
```

Mas cuidado com a robustez...

(veremos mais adiante)

OBRIGADO.

DÚVIDAS?